| | |
|---|---|
| **Started on** | Wednesday, 10 June 2026, 3:06 PM |
| **State** | Finished |
| **Completed on** | Wednesday, 10 June 2026, 5:28 PM |
| **Time taken** | 2 hours 21 mins |
| **Grade** | **14.00** out of 100.00 |

**Question 1**

Incorrect

Mark 0.00 out of 16.00

The `ComplexNumber` class is defined to represent complex numbers in the standard mathematical form: *a* + *bi*, where *a* is the real part and *b* is the imaginary part.

The class is partially implemented in the answer box below. Your task is to implement the following missing methods:

- `public static ComplexNumber add(ComplexNumber c1, ComplexNumber c2)`: Returns a new ComplexNumber that is the sum of c1 and c2.
- `public double magnitude()`: Calculates and returns the magnitude (absolute value) of the complex number.
- `public ComplexNumber conjugate()`: Returns a new ComplexNumber representing the complex conjugate (a - bi).

**Formulas:**
Addition: (a + bi) + (c + di) = (a + c) + (b + d)i
Magnitude: $\sqrt{(a^2 + b^2)}$
Conjugate of (a + bi) = a - bi

**For example:**

| Test | Result |
|---|---|
| `ComplexNumber c1 = new ComplexNumber(3.0, 4.0);`<br>`System.out.printf("%.2f\n", c1.magnitude());` | `5.00` |
| `ComplexNumber c1 = new ComplexNumber(1.5, 2.5);`<br>`ComplexNumber c2 = new ComplexNumber(2.0, -1.0);`<br>`ComplexNumber c3 = ComplexNumber.add(c1, c2);`<br>`System.out.println(c3.toString());` | `3.50 + 1.50i` |

**Answer:** (penalty regime: 0 %)

Reset answer

```
class ComplexNumber {
    private double real;
    private double imaginary;

    public ComplexNumber(double real, double imaginary) {
        this.real = real;
        this.imaginary = imaginary;
    }
    public double getReal() {
        return real;
    }
    public double getImaginary() {
        return imaginary;
    }
    // TODO: Implement the static add method
    public static ComplexNumber add(ComplexNumber c1, ComplexNumber c2) {
        double a1 = c1.getReal();
        double b1 = c1.getImaginary();
        double a2 = c2.getReal();
        double b2 = c2.getImaginary();
        double sum1 = 0;
        double sum2 = 0;
        sum1 = a1 + a2;
        sum2 = b1 + b2;
        return new ComplexNumber (sum1, sum2);
    }
    // TODO: Implement the magnitude method
    public double magnitude() {
        this.real = real;
        this.imaginary = imaginary;
        double magnitude = Math.sqrt(Math.pow(real, 2) + Math.pow(imaginary, 2));
        return magnitude;
    }
    // TODO: Implement the conjugate method
    public ComplexNumber conjugate() {
        this.real = real;
        this.imaginary = imaginary;
        imaginary = imaginary * (-1.0);
       return new ComplexNumber(real, imaginary) ;
```

```
    }
    // TODO: Implement the toString method
    public String toString() {

        return  "i";
    }
}
```

| | Test | Expected | Got | |
|---|---|---|---|---|
| ✔ | `ComplexNumber c1 = new ComplexNumber(3.0, 4.0);`<br>`System.out.printf("%.2f\n", c1.magnitude());` | `5.00` | `5.00` | ✔ |
| ✖ | `ComplexNumber c1 = new ComplexNumber(1.5, 2.5);`<br>`ComplexNumber c2 = new ComplexNumber(2.0, -1.0);`<br>`ComplexNumber c3 = ComplexNumber.add(c1, c2);`<br>`System.out.println(c3.toString());` | `3.50 + 1.50i` | `i` | ✖ |

Some hidden test cases failed, too.
Your code must pass all tests to earn any marks. Try again.

Show differences

**Show/hide question author's solution (Java)**

```
class ComplexNumber {
    private double real;
    private double imaginary;

    public ComplexNumber(double real, double imaginary) {
        this.real = real;
        this.imaginary = imaginary;
    }

    public double getReal() {
        return real;
    }

    public double getImaginary() {
        return imaginary;
    }

    public static ComplexNumber add(ComplexNumber c1, ComplexNumber c2) {
        return new ComplexNumber(c1.real + c2.real, c1.imaginary + c2.imaginary);
    }

    public double magnitude() {
        return Math.sqrt(real * real + imaginary * imaginary);
    }

    public ComplexNumber conjugate() {
        return new ComplexNumber(real, -imaginary);
    }

    public String toString() {
        if (imaginary >= 0) {
            return String.format("%.2f + %.2fi", real, imaginary);
        } else {
            return String.format("%.2f - %.2fi", real, -imaginary);
        }
    }
}
```

Incorrect

Marks for this submission: 0.00/16.00.

**Question 2**

Partially correct

Mark 14.00 out of 16.00

Consider the following Java class architecture demonstrating OOP paradigms (Interfaces, Abstract Classes, Inheritance, Polymorphism) provided in text format below:

**Class and Interface Structures:**

```java
// ========================================
// INTERFACE 1: Borrowable
// ========================================
interface Borrowable {
    void borrowItem();
    void returnItem();
}

// ========================================
// INTERFACE 2: Downloadable
// ========================================
interface Downloadable {
    void download();
}

// ========================================
// ABSTRACT CLASS: LibraryItem
// ========================================
abstract class LibraryItem {
    protected String id;
    protected String title;

    public LibraryItem(String id, String title) {
        this.id = id;
        this.title = title;
    }

    public String getId() {
        return id;
    }

    public String getTitle() {
        return title;
    }

    public abstract String getItemType();
}

// ========================================
// CONCRETE CLASS 1: PhysicalBook
// ========================================
class PhysicalBook extends LibraryItem implements Borrowable {
    private int pages;
    private String author;

    public PhysicalBook(String id, String title, int pages, String author) {
        super(id, title);
        this.pages = pages;
        this.author = author;
    }

    public int getPages() {
        return pages;
    }

    public String getAuthor() {
        return author;
    }

    @Override
    public void borrowItem() {
        System.out.println("Borrowing physical book: " + title);
    }

    @Override
    public void returnItem() {
        System.out.println("Returning physical book: " + title);
    }
```

```
    @Override
    public String getItemType() {
        return "Physical Book";
    }
}

// ========================================
// CONCRETE CLASS 2: EBook
// ========================================
class EBook extends LibraryItem implements Borrowable, Downloadable {
    private double fileSizeMB;
    private String format;

    public EBook(String id, String title, double fileSizeMB, String format) {
        super(id, title);
        this.fileSizeMB = fileSizeMB;
        this.format = format;
    }

    public double getFileSizeMB() {
        return fileSizeMB;
    }

    public String getFormat() {
        return format;
    }

    @Override
    public void borrowItem() {
        System.out.println("Checking out eBook license for: " + title);
    }

    @Override
    public void returnItem() {
        System.out.println("Returning eBook license for: " + title);
    }

    @Override
    public void download() {
        System.out.println("Downloading eBook: " + title + " [" + fileSizeMB + "MB, " + format + "]");
    }

    @Override
    public String getItemType() {
        return "Digital eBook";
    }
}
```

For each part below, write a single Java instruction that can be placed within the `Test` class's `main` method to perform the indicated task. Assume that all prior code executes successfully.

**Note:** The system accepts various spacing options, but you must ensure character spelling, capitalization, and double quotes are exact!

1. Write a statement to create a `PhysicalBook` object called `book1` with ID `"B01"`, title `"Java Basics"`, `500` pages, and author `"Alice"`.

PhysicalBook book1 = new PhysicalBook("B01", "Java Basics", 500, "A ✔

2. Write a statement to create an `EBook` object named `ebook1` with ID `"E01"`, title `"Advanced Java"`, file size `2.5`, and format `"PDF"`.

EBook ebook1 = new EBook("E01", "Advanced Java", 2.5 , "PDF"); ✖

3. **[Polymorphism]** Write a statement using a `LibraryItem` reference variable named `item1` to instantiate a new `PhysicalBook` object with ID `"B02"`, title `"OOP Concepts"`, `300` pages, and author `"Bob"`.

LibraryItem item1 = new PhysicalBook("B02", "OOP Concepts", 300, " ✔

4. **[Interface Polymorphism 1]** Write a statement using a `Borrowable` reference variable named `borrow1` to instantiate a new `EBook` object with ID `"E02"`, title `"Design Patterns"`, file size `1.8`, and format `"EPUB"`.

```
Borrowable borrow1 = new EBook("E02", "Design Patterns", 1.8, "EPU
```

5. **[Interface Polymorphism 2]** Write a statement using a `Downloadable` reference variable named `down1` to instantiate a new `EBook` object with ID `"E03"`, title `"Clean Code"`, file size `4.2`, and format `"PDF"`.

```
Downloadable down1 = new EBook("E03", "Clean Code", 4.2, "PDF");
```

6. Write an instruction to execute the borrow action on the `book1` object using its appropriate method. (Do not print).

```
book1.borrowItem();
```

7. Write an instruction to execute the download action on the `down1` object using its interface method. (Do not print).

```
down1.download();
```

8. Write an instruction to print the title of `item1` to the console and move the cursor to the next line, utilizing the parent class getter method.

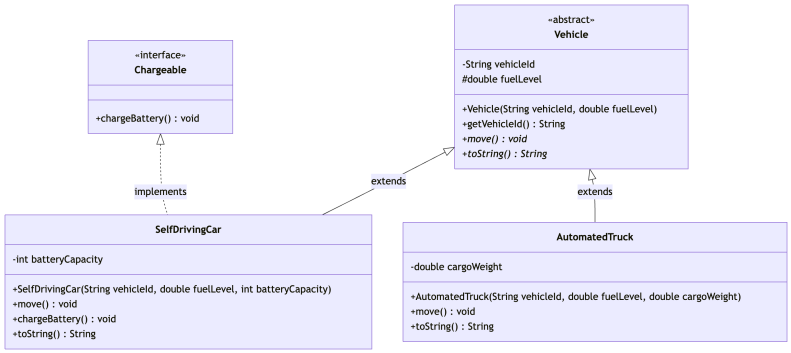
```
System.out.println(item1.getTitle());
```

**Question 3**

Incorrect

Mark 0.00 out of 30.00

Consider the architecture of an Autonomous Fleet Management system that includes interfaces, abstract implementations, and distinct specialization layers.




## Detailed OOP Implementation Rules:

1. `Chargeable` **(Interface):**
   - Declares an abstract method `void chargeBattery();`.
2. `Vehicle` **(Abstract Class):**
   - **Attributes:** `private String vehicleId`, `private double fuelLevel` (percentage value from `0.0` to `100.0`).
   - **Constructor:** `public Vehicle(String vehicleId, double fuelLevel)`
   - **Abstract Methods:** * `public abstract void move();` — Prints operational movement feedback.
     - `public abstract String toString();` — Overrides formatting metrics.
3. `SelfDrivingCar` **(Concrete Class):**
   - Inherits from `Vehicle` and implements `Chargeable`.
   - **Attributes:** `private int batteryCapacity` (in kWh).
   - Aggregates behavior by overriding `move()` to print: `"[Car <vehicleId>] Navigating autonomously in city lane. Fuel remaining: <fuelLevel>%"`
   - Implements `chargeBattery()` to set `fuelLevel` to `100.0` and prints: `"Charging lithium-ion pack to maximum capacity."`
4. `AutomatedTruck` **(Concrete Class):**
   - Inherits from `Vehicle`.
   - **Attributes:** `private double cargoWeight` (in tons).
   - Overrides `move()` to print: `"[Truck <vehicleId>] Hauling <cargoWeight> tons of cargo over highway network. Fuel remaining: <fuelLevel>%"`

Develop a complete **Java program that supports standard execution loops, mirrors this dynamic model, and matches the exact standard output behavior mapping shown** in the test samples below.

Note that the input is given all the time similar manner: you read a car and then a truck.

**HINT:** When reading composite string and double fields sequentially using Java Scanner utilities, be aware that primitive data tokens (`nextInt()`, `nextDouble()`) do not flush line-termination feeds from the stream pipeline automatically. If you construct sequential reads incorrectly, you will encounter stream alignment faults.

To secure stable pipeline alignment, use line-by-line streaming string capturing structures (`scanner.nextLine()`) and convert inputs using manual string parsing routines:

```
double fuel = Double.parseDouble(scanner.nextLine());
```

**For example:**

| Input | Result |
|---|---|
| Model-S<br>45.2<br>100<br>CyberTruck-X<br>88.5<br>7.5 | [Car Model-S] Navigating autonomously in city lane. Fuel remaining: 45.2%<br>[Truck CyberTruck-X] Hauling 7.5 tons of cargo over highway network. Fuel remaining: 88.5%<br>Charging lithium-ion pack to maximum capacity.<br><br>SYSTEM OVERVIEW:<br>Car Identity: Model-S \| Energy Status: 100.0% \| Pack Capacity: 100kWh<br>Truck Identity: CyberTruck-X \| Energy Status: 88.5% \| Cargo Weight: 7.5 tons |
| Bolt-EV<br>2.5<br>50<br>FreightLiner-Auto<br>10.2<br>40.0 | [Car Bolt-EV] Navigating autonomously in city lane. Fuel remaining: 2.5%<br>[Truck FreightLiner-Auto] Hauling 40.0 tons of cargo over highway network. Fuel remaining: 10.2%<br>Charging lithium-ion pack to maximum capacity.<br><br>SYSTEM OVERVIEW:<br>Car Identity: Bolt-EV \| Energy Status: 100.0% \| Pack Capacity: 50kWh<br>Truck Identity: FreightLiner-Auto \| Energy Status: 10.2% \| Cargo Weight: 40.0 tons |

**Answer:** (penalty regime: 0 %)

```
import java.util.Scanner;

interface ChargeAble{
    void chargeBattery();
}

abstract class Vehicle{
    private String vehicleId;
    private double fuelLevel;


    public Vehicle(String vehicleId, double fuelLevel){
       this.vehicleId = vehicleId;
       this.fuelLevel = fuelLevel;
    }

    public String getVehicleId(){
        return vehicleId;
    }

    public abstract void move();

    public abstract String toString();
}
class SelfDrivingCar extends Vehicle{
    private int batteryCapacity;

    public SelfDrivingCar(String vehicleId, double fuelLevel, int batteryCapacity){
        super(vehicleId, fuelLevel);
        this.batteryCapacity = batteryCapacity;


    }
    public void move(){

        System.out.println("[Car ]" + getVehicleId() + "Navigating autonomously in city lane. Fuel remaining: " +
        //"[Car <vehicleId>] Navigating autonomously in city lane. Fuel remaining: <fuelLevel>%"
    }
    // @Override
    public void chargeBattery(){

    }
    public String toString(){
        return "Charging lithium-ion pack to maximum capacity.";
    }
}

class AutomatedTruck extends Vehicle{
    private double cargoWeight;

    public AutomatedTruck(String vehicleId, double fuelLevel, double cargoWeight){
        super(vehicleId, fuelLevel);
        this.cargoWeight = cargoWeight;
    }

    public void move(){

    }
    @Override //???
```

```
    public String toString(){
        return "[Truck " + getVehicleId() + "] Hauling " + cargoWeight + "tons of cargo over highway network. Fue

    }
}

public class main{
    public static void main(String [] args){
        Scanner input = new Scanner(System.in);
        String car = input.nextLine();
        double fuelLevel = input.nextDouble();
        input.nextLine(); // buffer temizle
        int battery = input.nextInt();
        input.nextLine(); // buffer temizle

        String truck = input.nextLine();
        double fuelLevel2 = input.nextDouble();
        input.nextLine(); // buffer temizle
        int cargoWeight = input.nextInt();
        input.nextLine(); // buffer temizle


        SelfDrivingCar selfdriving = new SelfDrivingCar("car", fuelLevel, battery);


        if(fuelLevel == 100){
            selfdriving.toString();
        }
        else{
            System.out.println("Maximum");
        }

        AutomatedTruck automatedtruck = new AutomatedTruck ("truck", fuelLevel2, cargoWeight);

        automatedtruck.toString();
        System.out.println(automatedtruck.toString());
    }
}
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✖ | Model-S<br>45.2<br>100<br>CyberTruck-X<br>88.5<br>7.5 | [Car Model-S] Navigating autonomously in city lane. Fuel remaining: 45.2%<br>[Truck CyberTruck-X] Hauling 7.5 tons of cargo over highway network. Fuel remaining: 88.5%<br>Charging lithium-ion pack to maximum capacity.<br><br>SYSTEM OVERVIEW:<br>Car Identity: Model-S \| Energy Status: 100.0% \| Pack Capacity: 100kWh<br>Truck Identity: CyberTruck-X \| Energy Status: 88.5% \| Cargo Weight: 7.5 tons | ***Run error***<br>Exception in thread "main" java.util.InputMismatchException<br>at java.base/java.util.Scanner.throwFor(Scanner.java:947)<br>at java.base/java.util.Scanner.next(Scanner.java:1602)<br>at java.base/java.util.Scanner.nextInt(Scanner.java:2267)<br>at java.base/java.util.Scanner.nextInt(Scanner.java:2221)<br>at main.main(main.java:77) | ✖ |

Testing was aborted due to error.

Your code must pass all tests to earn any marks. Try again.

Show differences

**▾ Show/hide question author's solution (Java)**

```
import java.util.Scanner;

interface Chargeable {
    void chargeBattery();
}

abstract class Vehicle {
```

```
    private String vehicleId;
    protected double fuelLevel;

    public Vehicle(String vehicleId, double fuelLevel) {
        this.vehicleId = vehicleId;
        this.fuelLevel = fuelLevel;
    }

    public String getVehicleId() {
        return vehicleId;
    }

    public abstract void move();
    public abstract String toString();
}

class SelfDrivingCar extends Vehicle implements Chargeable {
    private int batteryCapacity;

    public SelfDrivingCar(String vehicleId, double fuelLevel, int batteryCapacity) {
        super(vehicleId, fuelLevel);
        this.batteryCapacity = batteryCapacity;
    }

    @Override
    public void move() {
        System.out.println("[Car " + getVehicleId() + "] Navigating autonomously in city lane. Fuel remaining: "
    }

    @Override
    public void chargeBattery() {
        this.fuelLevel = 100.0;
        System.out.println("Charging lithium-ion pack to maximum capacity.");
    }

    @Override
    public String toString() {
        return "Car Identity: " + getVehicleId() + " | Energy Status: " + fuelLevel + "% | Pack Capacity: " + bat
    }
}

class AutomatedTruck extends Vehicle {
    private double cargoWeight;

    public AutomatedTruck(String vehicleId, double fuelLevel, double cargoWeight) {
        super(vehicleId, fuelLevel);
        this.cargoWeight = cargoWeight;
    }

    @Override
    public void move() {
        System.out.println("[Truck " + getVehicleId() + "] Hauling " + cargoWeight + " tons of cargo over highway
    }

    @Override
    public String toString() {
        return "Truck Identity: " + getVehicleId() + " | Energy Status: " + fuelLevel + "% | Cargo Weight: " + ca
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        String carId = scanner.nextLine();
        double carFuel = Double.parseDouble(scanner.nextLine());
        int batteryCap = Integer.parseInt(scanner.nextLine());

        String truckId = scanner.nextLine();
        double truckFuel = Double.parseDouble(scanner.nextLine());
        double cargoW = Double.parseDouble(scanner.nextLine());

        SelfDrivingCar car = new SelfDrivingCar(carId, carFuel, batteryCap);
        AutomatedTruck truck = new AutomatedTruck(truckId, truckFuel, cargoW);

        car.move();
        truck.move();
        car.chargeBattery();

        System.out.println("\nSYSTEM OVERVIEW:");
```

```
88          System.out.println(car.toString());
89          System.out.println(truck.toString());
90
91          scanner.close();
92      }
93  }
```

Incorrect

Marks for this submission: 0.00/30.00.

**Question 4**

Not answered

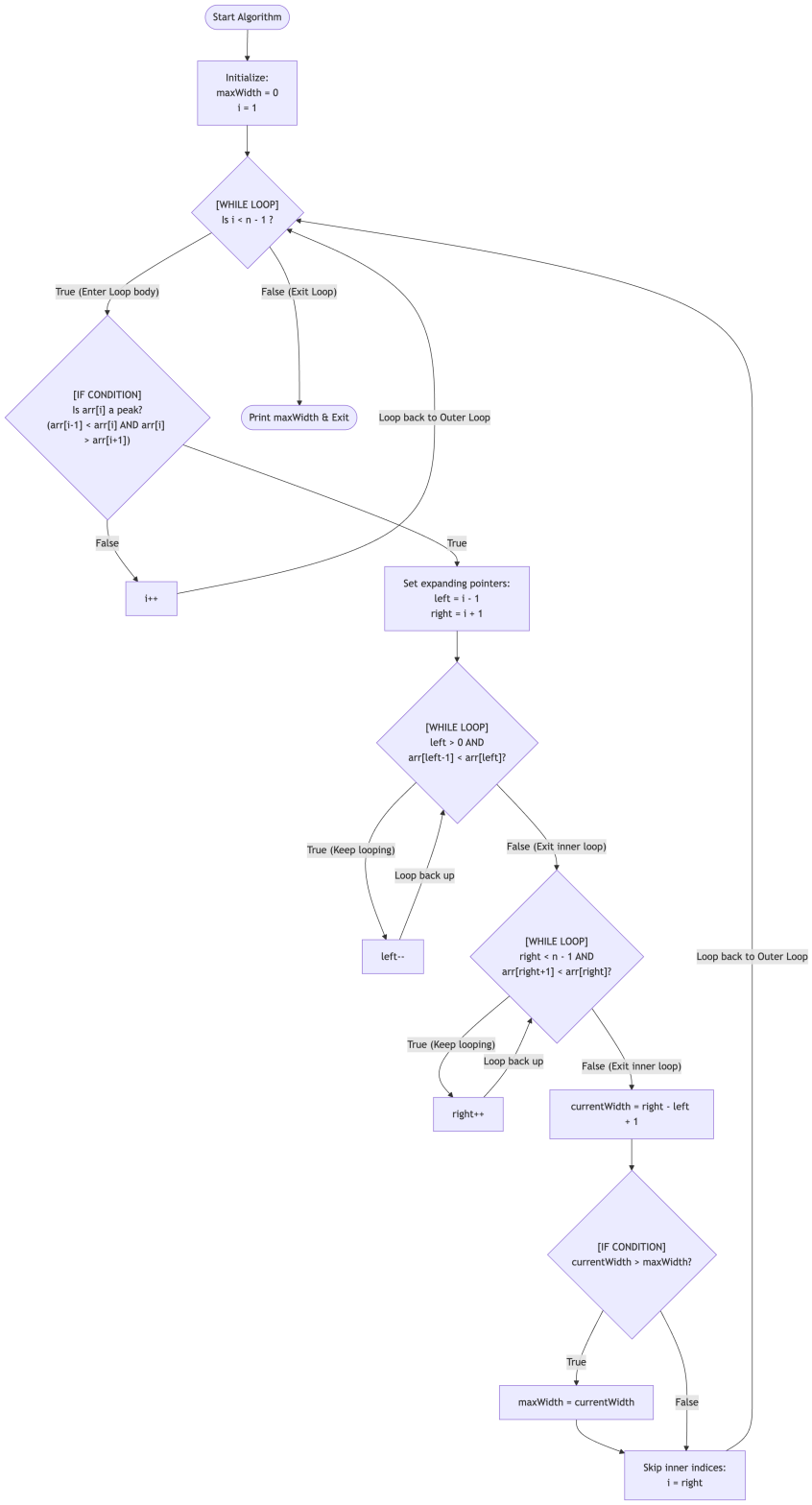
Mark 0.00 out of 18.00

You are given an algorithm to analyze digital signal waveforms and an array of **N** integers representing sampled voltage readings over time.

Your task is to find the width of the **widest signal peak** (often called a "mountain" in algorithms).

**Definition of a Signal Peak:**
A peak is a contiguous sequence of at least 3 samples where the voltage *strictly increases* to a singular peak index, and then *strictly decreases*.
For example, in the array `[2, 1, 4, 7, 3, 2, 5]`, the sequence `[1, 4, 7, 3, 2]` is a valid peak with a width of 5.

The algorithm flowchart is given as follows:




**Requirements:**

- You **must** use `while` loops to dynamically expand the left and right boundaries once a peak is detected.
- If there are no valid peaks (e.g., the signal is flat, constantly increasing, or constantly decreasing), output `0`.

**Input Format:**
Line 1: An integer **N** (the number of samples).
Line 2: **N** space-separated integers.

**Output Format:**

```
Widest Peak Width: X
```

**For example:**

| Input | Result |
|---|---|
| 7<br>2 1 4 7 3 2 5 | Widest Peak Width: 5 |
| 13<br>1 3 1 4 5 6 7 8 9 4 2 1 3 | Widest Peak Width: 10 |
| 5<br>10 9 8 7 6 | Widest Peak Width: 0 |
| 6<br>1 2 3 4 5 6 | Widest Peak Width: 0 |
| 7<br>2 2 2 3 2 2 2 | Widest Peak Width: 3 |
| 9<br>0 1 0 2 0 3 0 4 0 | Widest Peak Width: 3 |

**Answer:** (penalty regime: 0 %)

```
1
```

▾ **Show/hide question author's solution (Java)**

```
1 import java.util.Scanner;
2
3 public class Main {
```

```java
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        if (!scanner.hasNextInt()) return;

        int n = scanner.nextInt();
        int[] arr = new int[n];

        for (int i = 0; i < n; i++) {
            arr[i] = scanner.nextInt();
        }

        int maxWidth = 0;
        int i = 1;

        // Outer while loop to traverse the signal array looking for peaks
        while (i < n - 1) {
            // Check if the current point is the tip of a peak
            boolean isPeak = arr[i - 1] < arr[i] && arr[i] > arr[i + 1];

            if (!isPeak) {
                i++;
                continue;
            }

            int left = i - 1;
            int right = i + 1;

            // Inner while loop: expand down the left side of the peak safely
            while (left > 0 && arr[left - 1] < arr[left]) {
                left--;
            }

            // Inner while loop: expand down the right side of the peak safely
            while (right < n - 1 && arr[right + 1] < arr[right]) {
                right++;
            }

            // Calculate total width of this specific peak
            int currentWidth = right - left + 1;
            if (currentWidth > maxWidth) {
                maxWidth = currentWidth;
            }

            // Move the index to the right foot of the peak to skip redundant checks
            i = right;
        }

        System.out.println("Widest Peak Width: " + maxWidth);
        scanner.close();
    }
}
```

**Question 5**

Not answered

Mark 0.00 out of 20.00

Develop a complete **JAVA** program that decodes a compressed string containing nested repetition rules.

The compression format is given as: `k[encoded_string]`, where the `encoded_string` inside the square brackets is being repeated exactly `k` times. Note that `k` is guaranteed to be a positive integer, and the encoded strings can be deeply nested (e.g., `3[a2[c]]`).

**Assumptions:**
1. The input string is always valid; there are no extra white spaces, and square brackets are perfectly matched and well-formed.
2. The original data does not contain any digits. Digits are exclusively used for the repeat multiplier `k`.

**Technical Requirement:**
Your solution must effectively use String, StringBuilder, and ArrayList (as a structural stack) to dynamically manage the memory for string fragments and multiplier states.

**For example:**

| Input | Result |
|---|---|
| 3[a]2[bc] | aaabcbc |
| 3[a2[c]] | accaccacc |

**Answer:** (penalty regime: 0 %)

```
1
```

**Show/hide question author's solution (Java)**

```java
import java.util.Scanner;
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        if (!scanner.hasNextLine()) {
            return;
        }
        String s = scanner.nextLine();

        // Using ArrayLists as dynamic stacks to keep track of nested multipliers and string fragments
        ArrayList<Integer> countList = new ArrayList<>();
        ArrayList<StringBuilder> stringList = new ArrayList<>();

        StringBuilder currentString = new StringBuilder();
        int k = 0;

        for (char ch : s.toCharArray()) {
            if (Character.isDigit(ch)) {
                // Handle multi-digit numbers (e.g., 10, 100)
                k = k * 10 + (ch - '0');
            } else if (ch == '[') {
                // Push current states to our ArrayList 'stacks' and reset for the inner scope
                countList.add(k);
                stringList.add(currentString);
                currentString = new StringBuilder();
                k = 0;
            } else if (ch == ']') {
                // Pop the states and apply the multiplication
                StringBuilder decodedString = stringList.remove(stringList.size() - 1);
                int currentK = countList.remove(countList.size() - 1);

                for (int i = 0; i < currentK; i++) {
                    decodedString.append(currentString);
                }
                currentString = decodedString;
            } else {
                // Normal characters are just appended
                currentString.append(ch);
            }
        }

        System.out.println(currentString.toString());
        scanner.close();
    }
}
```